

LAB ASSIGNMENT – 13

NAME: CH VISHWANATH RAO

BATCH: 02

HT NO: 2303A51095

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple Repeated code blocks.

- Instructions:

- o Prompt AI to identify duplicate logic and replace it with functions or classes.

- o Ensure the refactored code maintains the same output.

- o Add docstrings to all functions.

- Sample Legacy Code:

Legacy script with repeated logic

- `print("Area of Rectangle:", 5 * 10)`
- `print("Perimeter of Rectangle:", 2 * (5 + 10))`
- `print("Area of Rectangle:", 7 * 12)`
- `print("Perimeter of Rectangle:", 2 * (7 + 12))`
- `print("Area of Rectangle:", 10 * 15)`
- `print("Perimeter of Rectangle:", 2 * (10 + 15))`

- Expected Output:

- o Refactored code with a reusable function and no duplication.

- o Well-documented code

CODE:

```
def validate_rectangle_dimensions(length, width):
    """
    Validate that rectangle dimensions are positive numbers.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    """
    if not isinstance(length, (int, float)) or not isinstance(width, (int, float)):
        raise ValueError("Length and width must be numeric values.")
    if length <= 0:
        raise ValueError("Length must be a positive number.")
    if width <= 0:
        raise ValueError("Width must be a positive number.")
    return True

def calculate_rectangle_area(length, width):
    """
    bool: True if dimensions are valid, False otherwise.
    """
    return length > 0 and width > 0
def calculate_rectangle_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.
    """
    return length * width
```

```
def calculate_rectangle_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The perimeter of the rectangle.
    """
    return 2 * (length + width)

def print_rectangle_properties(length, width):
    """
    Print the area and perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.
    """
    area = calculate_rectangle_area(length, width)
    perimeter = calculate_rectangle_perimeter(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")

# Main execution
if __name__ == "__main__":
    print_rectangle_properties(5, 10)
    print_rectangle_properties(7, 12)
    print_rectangle_properties(10, 15)
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/ass
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

- Instructions:

- o Identify repeated or related logic and extract it into reusable functions.

- o Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

Legacy script with inline repeated logic

- price = 250
- tax = price * 0.18
- total = price + tax
- print("Total Price:", total)
- price = 500
- tax = price * 0.18
- total = price + tax
- print("Total Price:", total)

- Expected Output:

- o Code with a function `calculate_total(price)` that can be reused for multiple price inputs.

- o Well-documented code

CODE:

```
def validate_cost(p):
    """Validate that the price is a non-negative number.
    Args:
        p (float): The price to validate.
    """
    if not isinstance(p, (int, float)):
        raise TypeError("Price must be a number.")
    if p < 0:
        raise ValueError("Price cannot be negative.")

def calculate_total_cost(p):
    """Calculate the total cost of an item with tax.
    Args:
        p (float): The price of the item.
    Returns:
        float: The total cost of the item with tax.
    """
    if not isinstance(p, (int, float)):
        raise TypeError("Price must be a number.")
    raise ValueError("Price cannot be negative.")

def calculate_total_cost(p):
    validate_cost(p)
    t = p * 0.18
    tot = p + t
    return tot

def handle_price(p):
    try:
        """Handle the price calculation and print the total cost.
        Args:
            p (float): The price of the item.
        """
        tp = calculate_total_cost(p)
        print(f"Total Price: {tp}")
    except (TypeError, ValueError) as e:
        print(f"Error: {e}")

handle_price(1200)
handle_price(600)
handle_price(700)
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai
Total Price: 1416.0
Total Price: 708.0
Total Price: 826.0
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code

principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator

2. Method Name: calculate_grade(self, marks)

3. The method must:

- o Accept marks as a parameter.
- o Return the corresponding grade as a string.
- o The grading logic must strictly follow the conditions below:

- Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"
- Marks $\geq 80 \rightarrow$ "Grade B"
- Marks $\geq 70 \rightarrow$ "Grade C"
- Marks $\geq 40 \rightarrow$ "Grade D"
- Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

- o The class
- o The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

- Given code:

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
```

```
    print("Grade C")
```

```
marks = 72
```

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

CODE:

```
1 class GradeCalculator:
2     def __init__(self, m):
3         self.m = m
4
5     def calculate_grade(self):
6         if not isinstance(self.m, (int, float)):
7             raise TypeError("Marks must be a number.")
8         if self.m < 0 or self.m > 100:
9             raise ValueError("Marks must be between 0 and 100.")
10
11         if self.m >= 90:
12             return "Grade A"
13         elif self.m >= 80:
14             return "Grade B"
15         elif self.m >= 70:
16             return "Grade C"
17         elif self.m >= 40:
18             return "Grade D"
19         else:
20             return "Fail"
21
22 test_marks = [85, 72, 35, 95]
23
24 for m in test_marks:
25     try:
26         gc = GradeCalculator(m)
27         g = gc.calculate_grade()
28         print(f"Marks: {m} -> {g}")
29     except (TypeError, ValueError) as e:
30         print(f"Error: {e}")
```

OUTPUT:

```
Marks: 85 -> Grade B
Marks: 72 -> Grade C
Marks: 35 -> Fail
Marks: 95 -> Grade A
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```

- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.

CODE:

```
def get_input():
    try:
        value = int(input("Enter number: "))
        return value
    except ValueError:
        print("Invalid input. Please enter a valid integer.")
        return None

def process_input(value):
    if value is not None:
        return value ** 2
    return None

def display_output(output):
    if output is not None:
        print(f"The square of the number is: {output}")
    else:
        print("No valid result to display.")

def main():
    value = get_input()
    output = process_input(value)
    display_output(output)

if __name__ == "__main__":
    main()
    # Additional test cases to verify the square calculation function
    def test_square_calculation():
        """Test the square calculation with various inputs."""
        test_values = [5, 10, 0, -3]
        for num in test_values:
            result = process_input(num)
            display_output(result)
    test_square_calculation()
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assggg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assg
Enter number: 1200
The square of the number is: 1440000
The square of the number is: 25
The square of the number is: 100
The square of the number is: 0
The square of the number is: 9
```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

CODE:

```
class SalaryCalculator:
    def __init__(self, s):
        self._s = s

    def _validate_salary(self):
        """Validate salary value."""
        if not isinstance(self._s, (int, float)):
            raise ValueError("Salary must be a number.")
        if self._s < 0:
            raise ValueError("Salary must be non-negative.")

    def calculate_tax(self):
        """Calculate tax on salary."""
        self._validate_salary()
        return self._s * 0.2

    def calculate_net_salary(self):
        """Calculate net salary after tax."""
        t = self.calculate_tax()
        return self._s - t

    def display_salary_info(self):
        """Display salary information."""
        try:
            t = self.calculate_tax()
            n = self.calculate_net_salary()
            print(f"Gross Salary: {self._s}")
            print(f"Tax (20%): {t}")
            print(f"Net Salary: {n}")
        except ValueError as e:
            print(f"Error: {e}")

sc = SalaryCalculator(50000)
sc.display_salary_info()
sc = SalaryCalculator(75000)
sc.display_salary_info()
sc = SalaryCalculator(60000)
sc.display_salary_info()
```

OUTPUT:

```
Gross Salary: 50000
Tax (20%): 10000.0
Net Salary: 40000.0
```


Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity
- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:
    if u == name:
        found = True

print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification

CODE:

```
172
173 # Using a set for O(1) average-case lookup instead of O(n) list search
174 users = {"admin", "guest", "editor", "viewer"}
175 name = input("Enter username: ")
176 found = name in users
177 print("Access Granted" if found else "Access Denied")
178
```

OUTPUT:

```
Enter username: admin
Access Granted
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions

- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

o add_book(title, author, isbn)

o remove_book(isbn)

o search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.

Given Code

Library Management System (Unstructured Version)

This code needs refactoring into a proper module with documentation.

```
library_db = {}
```

Adding first book

```
title = "Python Basics"
```

```
author = "John Doe"
```

```
isbn = "101"
```

```
if isbn not in library_db:
```

```
    library_db[isbn] = {"title": title, "author": author}
```

```
    print("Book added successfully.")
```

```
else:
```

```
    print("Book already exists.")
```

Adding second book (duplicate logic)

```
title = "AI Fundamentals"
```

```
author = "Jane Smith"
```

```
isbn = "102"
```

```
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
```

CODE:

```
db = {}

def add(t, a, i):
    if not t or not a or not i:
        raise ValueError("Title, author, and ISBN cannot be empty.")

    if i not in db:
        db[i] = {'t': t, 'a': a}
        print("Book added successfully.")
        return True
    else:
        print("Book already exists.")
        return False

def remove(i):
    if i in db:
        del db[i]
        print("Book removed successfully.")
        return True
    else:
        print("Book not found.")
        return False

def search(i):
    if i in db:
        print("Book Found:", db[i])
        return db[i]
    else:
        print("Book not found.")
        return None

# Usage
if __name__ == "__main__":
    add("Python Basics", "John Doe", "101")
    add("AI Fundamentals", "Jane Smith", "102")
    search("101")
    remove("101")
    search("101")
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive
Book added successfully.
Book added successfully.
Book Found: {'t': 'Python Basics', 'a': 'John Doe'}
Book removed successfully.
Book not found.
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg>
```

Functions

add_book(title, author, isbn)

Add a new book to the library database.

Args:

title (str): The title of the book.

author (str): The author of the book.

isbn (str): The ISBN of the book (unique identifier).

Returns:

bool: True if book added successfully, False if book already exists.

Raises:

ValueError: If any parameter is empty or invalid.

remove_book(isbn)

Remove a book from the library database by ISBN.

Args:

isbn (str): The ISBN of the book to remove.

Returns:

bool: True if book removed successfully, False if book not found.

search_book(isbn)

Search for a book in the library database by ISBN.

Args:

isbn (str): The ISBN of the book to search.

Returns:

dict: Book details (title, author) if found, None otherwise.

Data

```
library_db = {}
```

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive
Book added successfully.
Book added successfully.
Book Found: {'t': 'Python Basics', 'a': 'John Doe'}
Book removed successfully.
Book not found.
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg>
```

Task 8– Fibonacci Generator

Write a program to generate the Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

fibonacci bad version

n=int(input("Enter limit: "))

a=0

b=1

print(a)

print(b)

for i in range(2,n):

c=a+b

print(c)

a=b

b=c

CODE:

```
def generate_fibonacci(n):  
    if n <= 0:  
        return []  
    elif n == 1:  
        return [0]  
    elif n == 2:  
        return [0, 1]  
  
    fib_sequence = [0, 1]  
    for i in range(2, n):  
        fib_sequence.append(fib_sequence[i-1] + fib_sequence[i-2])  
  
    return fib_sequence  
  
number = int(input("Enter limit: "))  
fib_numbers = generate_fibonacci(number)  
for num in fib_numbers:  
    print(num)
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/demo.py
Enter limit: 10
0
1
1
2
3
5
8
13
21
34
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version

a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1
fb=0
for i in range(2,b):
    if b%i==0:
        fb=1
if fa==0 and fb==0 and abs(a-b)==2:
    print("Twin Primes")
else:
    print("Not Twin Primes")
```

CODE:

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def is_twin_prime(p1, p2):
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2

def generate_twin_primes(s, e):
    tp = []
    for i in range(s, e + 1):
        if is_twin_prime(i, i + 2):
            tp.append((i, i + 2))
    return tp

# example usage with shorter vars
x = 11
y = 13
if is_twin_prime(x, y):
    print(f"{x} and {y} are twin primes.")
else:
    print(f"{x} and {y} are not twin primes.")

s = 1
e = 100
lst = generate_twin_primes(s, e)
print(f"Twin primes between {s} and {e}: {lst}")
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg
11 and 13 are twin primes.
Twin primes between 1 and 100: [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg
33 and 56 are not twin primes.
Twin primes between 1 and 100: [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg>
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat

2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
```

```
if year % 12 == 0:
```

```
    print("Monkey")
```

```
elif year % 12 == 1:
```

```
    print("Rooster")
```

```
elif year % 12 == 2:
```

```
    print("Dog")
```

```
elif year % 12 == 3:
```

```
    print("Pig")
```

```
elif year % 12 == 4:
```

```
    print("Rat")
```

```
elif year % 12 == 5:
```

```
    print("Ox")
```

```
elif year % 12 == 6:  
    print("Tiger")  
elif year % 12 == 7:  
    print("Rabbit")  
elif year % 12 == 8:  
    print("Dragon")
```

```
elif year % 12 == 9:  
    print("Snake")  
elif year % 12 == 10:  
    print("Horse")  
elif year % 12 == 11:  
    print("Goat")
```

You must:

1. Create a reusable function: `get_zodiac(year)`
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

CODE:

```
#refactor the code as per the instructions 1. Create a reusable function: get_zodiac(year)2. Replace the if-elif chain with a cleaner structure (e.g., list ordictionary).3. Add proper docstrings.4. Separate input handling from logic

def get_chinese_zodiac(year):
    """Get the Chinese zodiac sign for a given year.

    Args:
        year (int): The year to check.

    Returns:
        str: The Chinese zodiac sign.
    """
    zodiac_animals = [
        "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
        "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
    ]
    return zodiac_animals[(year % 12)]

def main():
    """Main function to handle user input and display the zodiac sign."""
    try:
        input_year = int(input("Enter a year: "))
        zodiac_sign = get_chinese_zodiac(input_year)
        print(f"The Chinese zodiac sign for the year {input_year} is: {zodiac_sign}")
    except ValueError:
        print("Invalid input. Please enter a valid integer for the year.")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/demo.py
Enter a year: 2004
The Chinese zodiac sign for the year 2004 is: Monkey
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/demo.py
Enter a year: 2023
The Chinese zodiac sign for the year 2023 is: Rabbit
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/demo.py
Enter a year: 2026
The Chinese zodiac sign for the year 2026 is: Horse
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> █
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$ (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values

- Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker (Unstructured Version)

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum_digits = sum_digits + digit
```

```
    temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```
        print("True")
```

```
    else:
```

```
        print("False")
```

```
else:
```

```
    print("False")
```

You must:

1. Create a reusable function: `is_harshad(number)`
2. The function must:
 - o Accept an integer parameter.
 - o Return True if the number is divisible by the sum of its digits.
 - o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

CODE:

```
def is_harshad(input_number):
    if input_number <= 0:
        return False
    remaining_value = input_number
    accumulated_digit_sum = 0
    while remaining_value > 0:
        current_digit = remaining_value % 10
        accumulated_digit_sum += current_digit
        remaining_value //= 10
    return accumulated_digit_sum != 0 and input_number % accumulated_digit_sum == 0

def main():
    try:
        user_supplied_value = int(input("Enter a number: "))
        if is_harshad(user_supplied_value):
            print("True")
        else:
            print("False")
    except ValueError:
        print("Invalid input. Please enter a valid integer.")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/de
Enter a number: 200
True
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/de
Enter a number: 14
False
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/de
Enter a number: 10
True
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: "))
```

```
fact = 1
```

```

i = 1

while i <= n:

    fact = fact * i

    i = i + 1

count = 0

while fact % 10 == 0:

    count = count + 1

    fact = fact // 10

print("Trailing zeros:", count)

```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`
2. The function must:
 - o Accept a non-negative integer `n`.
 - o Return the number of trailing zeros in `n!`.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

CODE:

```

def count_trailing_zeros(factorial_num):
    """
    Count the number of trailing zeros in factorial_num!.
    """
    if factorial_num < 0:
        raise ValueError("Input must be a non-negative integer.")

    trailing_zero_count = 0
    divisor = 5

    while factorial_num >= divisor:
        trailing_zero_count += factorial_num // divisor
        divisor *= 5

    return trailing_zero_count

def main():
    try:
        user_input = int(input("Enter a non-negative integer: "))
        result = count_trailing_zeros(user_input)
        print(f"The number of trailing zeros in {user_input}! is: {result}")
    except ValueError as error:
        print(f"Error: {error}")

if __name__ == "__main__":
    main()

```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/1
Enter a non-negative integer: 140
The number of trailing zeros in 140! is: 34
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/1
Enter a non-negative integer: 123
The number of trailing zeros in 123! is: 28
```

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
- Edge: $1 \rightarrow [1]$
- Negative: -5
- Large: 27 (well-known long sequence)
- Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - o If n is even $\rightarrow \text{next} = n / 2$.
 - o If n is odd $\rightarrow \text{next} = 3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)

- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)
- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

CODE:

```
def generate_collatz_sequence(num):
    if not isinstance(num, int) or num <= 0:
        raise ValueError("Input must be a positive integer.")

    seq = []

    while num != 1:
        seq.append(num)
        if num % 2 == 0:
            num //= 2
        else:
            num = 3 * num + 1

    seq.append(1)
    return seq

def main():
    try:
        user_input = int(input("Enter a positive integer: "))
        collatz_seq = generate_collatz_sequence(user_input)
        print(f"Collatz sequence for {user_input}: {collatz_seq}")
    except ValueError as e:
        print(f"Error: {e}")

if __name__ == "__main__":
    main()

# Test cases
assert generate_collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1]
assert generate_collatz_sequence(19) == [19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
assert generate_collatz_sequence(1) == [1]
try:
    generate_collatz_sequence(0)
    assert False, "Should raise ValueError"
except ValueError:
    pass
```

OUTPUT:

```
C:\Users\chvis\OneDrive\Desktop\ai\assgg & C:\Python314\python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_11.py/demo.py
Enter a positive integer: 120
Collatz sequence for 120: [120, 60, 30, 15, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1]
C:\Users\chvis\OneDrive\Desktop\ai\assgg & C:\Python314\python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_11.py/demo.py
Enter a positive integer: 19
Collatz sequence for 19: [19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
C:\Users\chvis\OneDrive\Desktop\ai\assgg & C:\Python314\python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_11.py/demo.py
Enter a positive integer: 0
Traceback (most recent call last):
  File "c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_11.py/demo.py", line 10, in generate_collatz_sequence
    raise ValueError("Input must be a positive integer.")
ValueError: Input must be a positive integer.
```


Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:
- Normal: 5 $\rightarrow$ [2, 1, 3, 4, 7]
- Edge: 1 $\rightarrow$ [2]
- Negative: -5 $\rightarrow$ Error
- Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

CODE:

```
def generate_lucas_sequence(n):
    if not isinstance(n, int):
        raise TypeError("Input must be an integer.")
    if n <= 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    a, b = 2, 1

    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b

    return sequence

# Test cases using pytest
def test_generate_lucas_normal():
    """Test normal case with 5 terms"""
    assert generate_lucas_sequence(5) == [2, 1, 3, 4, 7]

def test_generate_lucas_single_term():
    """Test edge case with 1 term"""
    assert generate_lucas_sequence(1) == [2]

def test_generate_lucas_large():
    """Test with 10 terms"""
    assert generate_lucas_sequence(10) == [2, 1, 3, 4, 7, 11, 18, 29, 47, 76]

def test_generate_lucas_negative():
    """Test error handling for negative input"""
    with pytest.raises(ValueError):
        generate_lucas_sequence(-5)

def test_generate_lucas_zero():
    """Test error handling for zero input"""
    with pytest.raises(ValueError):
        generate_lucas_sequence(0)

def test_generate_lucas_non_integer():
    """Test error handling for non-integer input"""
    with pytest.raises(TypeError):
        generate_lucas_sequence(5.5)

def test_generate_lucas_two_terms():
    """Test with 2 terms"""
    assert generate_lucas_sequence(2) == [2, 1]

if __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/demo.py
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 -- C:/Python314/python.exe
cachedir: .pytest_cache
rootdir: C:\Users\chvis\OneDrive\Desktop\ai\assgg
collected 7 items

lab_13.py/demo.py::test_generate_lucas_normal PASSED
lab_13.py/demo.py::test_generate_lucas_single_term PASSED
lab_13.py/demo.py::test_generate_lucas_large PASSED
lab_13.py/demo.py::test_generate_lucas_negative PASSED
lab_13.py/demo.py::test_generate_lucas_zero PASSED
lab_13.py/demo.py::test_generate_lucas_non_integer PASSED
lab_13.py/demo.py::test_generate_lucas_two_terms PASSED

7 passed (in 0.01s)
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.
- Test Cases to Design:
- Normal: "hello" → (2,3)
- Edge: "" → (0,0)
- Only vowels: "aeiou" → (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

CODE:

```
# Analyze a string to count vowels and consonants.
# Test Cases:
# - Normal case: "hello" should return (2, 3)
# - Empty string: "" should return (0, 0)
# - All vowels: "aeiou" should return (5, 0)
# - Long text: Verify with extended content
# - Validation: Use pytest for correctness

def count_vowels_consonants(input_string):
    vowel_set = set("aeiouAEIOU")
    vowel_count = 0
    consonant_count = 0

    for character in input_string:
        if character.isalpha():
            if character in vowel_set:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count

# Test cases
assert count_vowels_consonants("hello") == (2, 3)
assert count_vowels_consonants("") == (0, 0)
assert count_vowels_consonants("aeiou") == (5, 0)
assert count_vowels_consonants("The quick brown fox jumps over the lazy dog") == (11, 24)
print("All tests passed!")
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_13.py/demo.py
All tests passed!
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg> █
```